23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : C5
Team Members :

Roll No	Student Name
AM.SC.U4CSE25201	AKSHANYA S
AM.SC.U4CSE25229	KRISHNA JAYAKUMAR
AM.SC.U4CSE25215	DHARUN A

Project Title : Smart Campus Resource Booking System

CONTENTS

Sl. No	Chapter	Page No
1	Abstract	
2	Introduction	
3	Requirements (Functional & Non-functional)	
4	UI Design	
5	Database Design	
6	Database Selection & Justification	
7	Implementation	
8	Testing & Validation	
9	Results & Screenshots	
10	Challenges faced & solutions	
11	Conclusion & future enhancements	
	References	

1. Abstract

Campus facilities such as classrooms, laboratories, study rooms, sports venues, equipment, and parking spaces are often managed through manual registers, leading to scheduling conflicts, low utilization, and administrative inefficiencies. The proposed Smart Campus Resource Booking System is a centralized platform that enables efficient, conflict-free reservation and management of shared campus facilities through real-time availability tracking and automated scheduling. By utilizing a relational PostgreSQL database to manage users, resources, reservations, maintenance, and notifications, the system provides a unified interface for students, faculty, and administrators. This project demonstrates advanced database concepts like transaction concurrency and strict constraint validation, ensuring fair allocation and an organized approach to campus infrastructure management.

2. Introduction

Background With the continuous growth of academic and extracurricular activities on campus, managing shared infrastructure effectively has become a critical operational requirement. The campus houses a wide variety of shared spaces and assets, ranging from high-capacity lecture halls and specialized computer labs to sports complexes and individual equipment. Ensuring that these resources are fairly accessible to students and faculty while maintaining proper oversight is a complex logistical challenge.

Existing Scenario and Limitations Currently, campus resources are primarily tracked through fragmented systems or manual logbooks. This existing approach has several severe limitations. First, the lack of a centralized, real-time database means users cannot easily check if a room or lab is available without physically visiting the location or contacting administrative staff. Second, manual tracking frequently results in double bookings—where two different groups reserve the same classroom or equipment for overlapping time slots—causing friction and wasted time. Finally, administrative workflows, such as approving requests, scheduling maintenance blocks, and maintaining historical records, are tedious, error-prone, and lack proper analytics to understand resource utilization.

Proposed System and Problem Resolution The Smart Campus Resource Booking System is proposed to directly address these bottlenecks by digitizing the entire reservation lifecycle. By implementing a robust relational database backend, the system offers a single source of truth for resource availability. It shifts the workflow from a manual, trust-based system to an automated, rule-based environment. The database actively prevents overlapping bookings, automatically handles waitlists if a slot frees up, and enforces role-based access so that only authorized personnel can approve requests or schedule maintenance. Ultimately, this application improves transparency, reduces administrative overhead, and maximizes the practical utilization of campus facilities.

3. Requirements (Functional & Non-functional)

Functional Requirements

Requirement Type	Description
Authentication	Secure login system enforcing role-based access control (RBAC) specifically tailored for Students, Faculty, Administrators, and Maintenance staff.
Resource Discovery	Users must be able to search for specific resources and view their real-time availability status ('Available' or 'Under Maintenance').
Booking Management	The system must allow users to create, modify, and cancel reservations for specific time slots.
Conflict Resolution	Automated database-level scheduling logic must actively prevent double bookings for the same resource during overlapping times.
Waitlist System	Users can opt to join a waitlist for booked slots and will be automatically promoted/notified if the previous booking is cancelled.
Approval Workflow	Specific resources require an administrative approval step, transitioning from 'Pending' to 'Approved' or 'Rejected'.
Maintenance Blocks	Authorized staff can block out resources for scheduled maintenance, preventing any user bookings during that period.

Non-functional Requirements

Requirement Type	Description

Concurrency	The database must seamlessly handle simultaneous booking attempts from multiple users without violating data integrity or allowing race conditions.
Usability	The web interface should be highly intuitive, utilizing clear visual cues (like color-coded availability badges) so users can navigate it with zero training.
Performance	Availability queries (e.g., checking if a lab is free tomorrow at 10 AM) must execute instantly, leveraging proper database indexing.
Scalability	The relational schema must be flexible enough to accommodate the addition of new resource categories or an expanding user base without structural changes.

4. UI Design

The user interface of the Smart Campus Resource Booking System is designed with simplicity and efficiency in mind, built using clean HTML and modern CSS. We focused on creating a dashboard that provides immediate visual feedback regarding resource availability.

Navigation and Layout The application features a sticky navigation bar that adapts based on the user's role, ensuring students only see their reservations, while administrators have access to an administrative panel. The main dashboard is dominated by a hero section where users can quickly input their required resource, start time, end time, and opt into a waitlist.

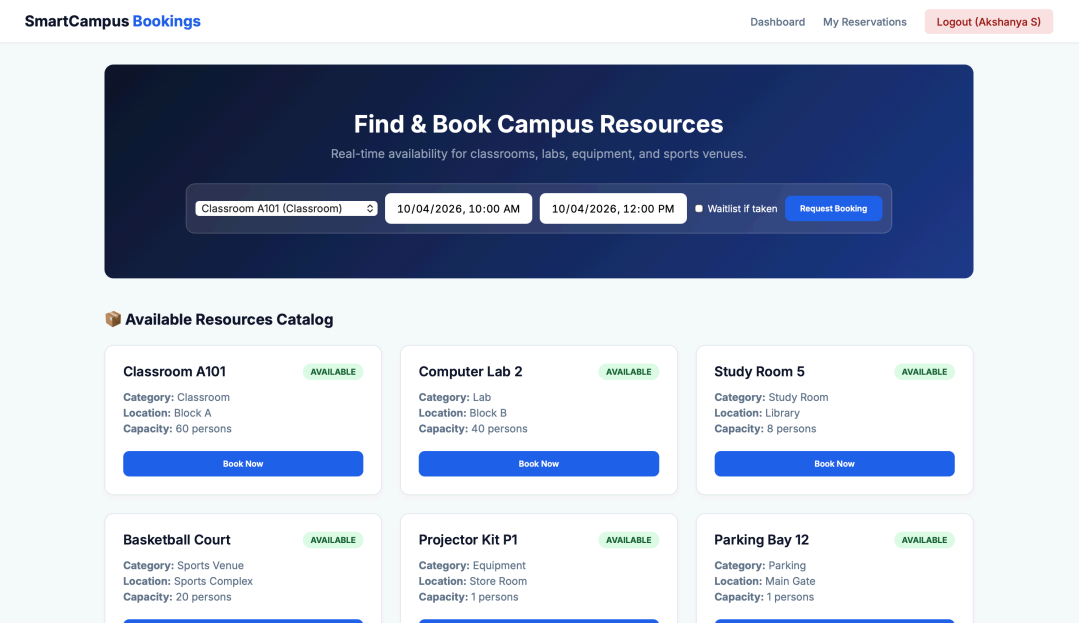


Figure 4.1 - Main user dashboard showcasing real-time resource availability and search functionality.

Below the search bar, the resource catalog is presented in a responsive grid layout. Each facility is displayed as a card containing its category, location, and capacity. To enhance usability, we implemented dynamic status badges. If a resource is free, a green 'Available' badge is shown alongside an active 'Book Now' button. If it is booked or under repair, a red 'Under Maintenance' badge appears, and the booking button is disabled to prevent user error.

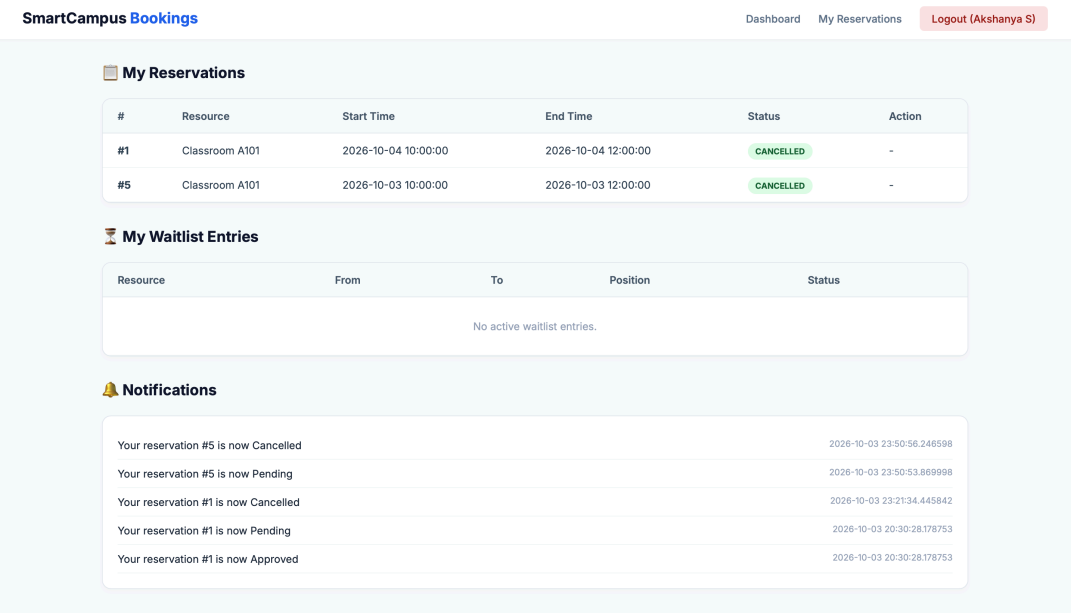


Figure 4.2 - The "My Reservations" page where users can track booking status, view waitlist positions, and manage cancellations.

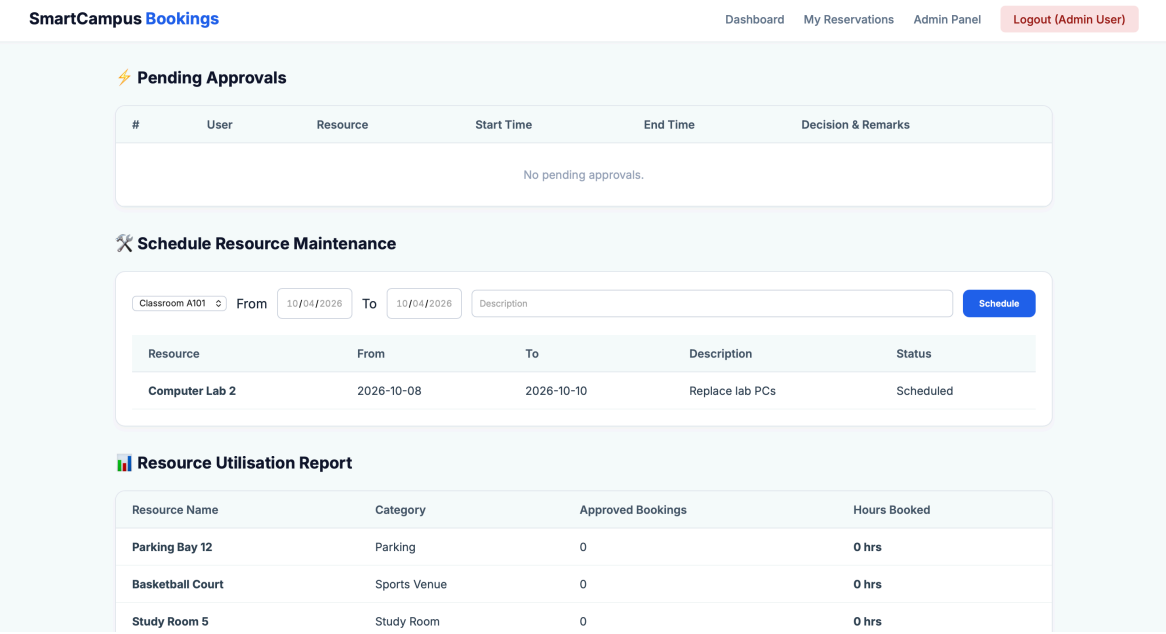


Figure 4.3 - Administrative dashboard displaying pending approvals, maintenance scheduling inputs, and resource utilization reports.

5. Database Design

The foundation of the system is a robust relational database designed to accurately track users, complex resource states, and transaction histories.

Entity-Relationship (ER) Modeling

The ER diagram maps out the core entities and their relationships. A USER is linked to exactly one ROLE, which governs their permissions. RESOURCES belong to a RESOURCE_CATEGORY and can undergo multiple MAINTENANCE_RECORDS. The central entity is RESERVATION, which bridges a USER to a RESOURCE for a specific time frame. A vital business rule depicted is the APPROVAL process: because a rejected reservation can be resubmitted, a single reservation can have a one-to-many relationship with approval records over time.’

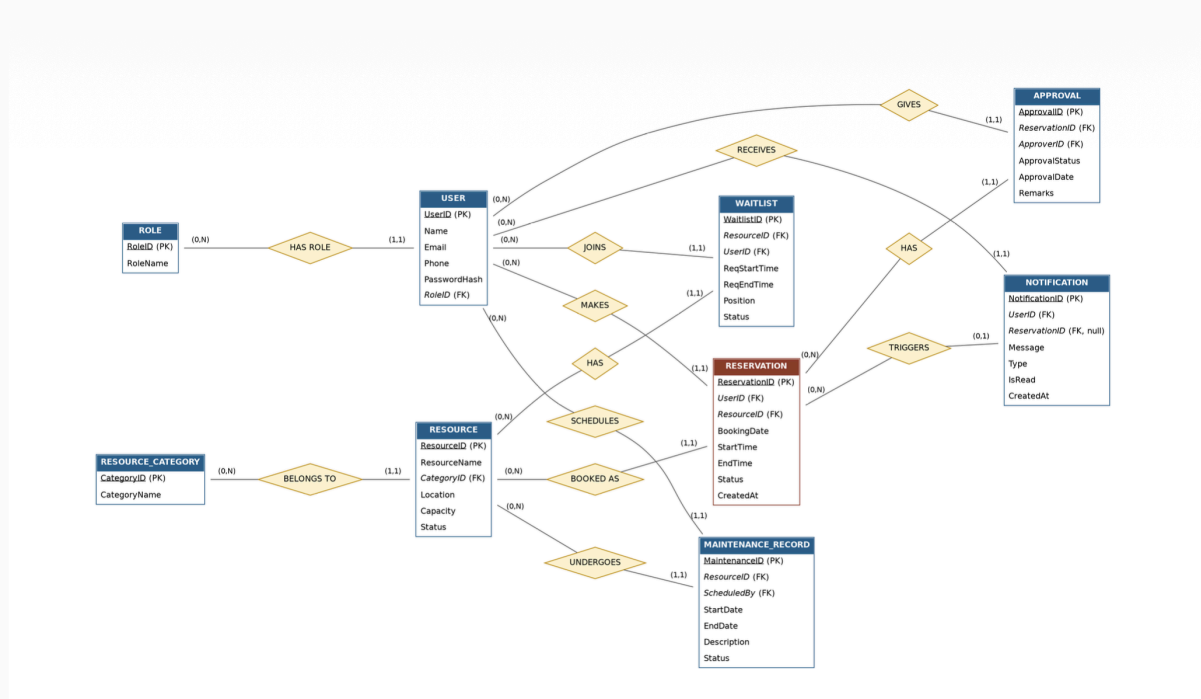


Figure 5.1 - Entity-Relationship Diagram detailing cardinality and participation constraints.

Relational Schema

The ER model was systematically converted into the following relational tables, enforcing primary keys (PK) and foreign keys (FK) to maintain referential integrity.

```
ROLE (RoleID, RoleName)

USER (UserID, Name, Email, Phone, Password, RoleID*)

RESOURCE_CATEGORY (CategoryID, CategoryName)

RESOURCE (ResourceID, ResourceName, CategoryID*, Location,
Capacity, Status)

RESERVATION (ReservationID, UserID*, ResourceID*, BookingDate,
StartTime, EndTime, Status, CreatedAt)

APPROVAL (ApprovalID, ReservationID*, ApproverID*,
ApprovalStatus, ApprovalDate, Remarks)

WAITLIST (WaitlistID, ResourceID*, UserID*, ReqStartTime,
ReqEndTime, Position, Status)

MAINTENANCE_RECORD (MaintenanceID, ResourceID*, ScheduledBy*,
StartDate, EndDate, Description, Status)

NOTIFICATION (NotificationID, UserID*, ReservationID*,
Message, Type, IsRead, CreatedAt)
```

(* = foreign key; primary key of each relation is underlined below.)

Column	Data Type	Constraint
ROLE		
RoleID	SERIAL	PRIMARY KEY
RoleName	VARCHAR(50)	NOT NULL, UNIQUE
USER		
UserID	SERIAL	PRIMARY KEY
Name	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	NOT NULL, UNIQUE
Phone	VARCHAR(15)	—
Password	VARCHAR(255)	NOT NULL
RoleID	INT	NOT NULL, FK → ROLE(RoleID)
RESOURCE_CATEGORY		
CategoryID	SERIAL	PRIMARY KEY
CategoryName	VARCHAR(50)	NOT NULL, UNIQUE
RESOURCE		

ResourceID	SERIAL	PRIMARY KEY
ResourceName	VARCHAR(100)	NOT NULL
CategoryID	INT	NOT NULL, FK → RESOURCE_CATEGORY
Location	VARCHAR(100)	—
Capacity	INT	CHECK (Capacity > 0)
Status	VARCHAR(20)	CHECK IN ('Available','Under Maintenance','Inactive'), DEFAULT 'Available'
RESERVATION		
ReservationID	SERIAL	PRIMARY KEY
UserID	INT	NOT NULL, FK → USER
ResourceID	INT	NOT NULL, FK → RESOURCE
BookingDate	DATE	NOT NULL
StartTime	TIMESTAMP	NOT NULL
EndTime	TIMESTAMP	NOT NULL, CHECK (EndTime > StartTime)
Status	VARCHAR(20)	CHECK IN ('Pending','Approved','Rejected','Cancelled','Completed')
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
APPROVAL		
ApprovalID	SERIAL	PRIMARY KEY
ReservationID	INT	NOT NULL, FK → RESERVATION
ApproverID	INT	NOT NULL, FK → USER
ApprovalStatus	VARCHAR(20)	CHECK IN ('Approved','Rejected','Pending')
ApprovalDate	TIMESTAMP	—
Remarks	VARCHAR(255)	—
WAITLIST		
WaitlistID	SERIAL	PRIMARY KEY
ResourceID	INT	NOT NULL, FK → RESOURCE
UserID	INT	NOT NULL, FK → USER
ReqStartTime	TIMESTAMP	NOT NULL
ReqEndTime	TIMESTAMP	NOT NULL, CHECK (> ReqStartTime)
Position	INT	NOT NULL
Status	VARCHAR(20)	CHECK IN ('Waiting','Notified','Expired','Converted')
MAINTENANCE_RECORD		
MaintenanceID	SERIAL	PRIMARY KEY
ResourceID	INT	NOT NULL, FK → RESOURCE
ScheduledBy	INT	NOT NULL, FK → USER
StartDate	DATE	NOT NULL
EndDate	DATE	NOT NULL, CHECK (>= StartDate)
Description	VARCHAR(255)	—
Status	VARCHAR(20)	CHECK IN ('Scheduled','In Progress','Completed','Cancelled')

NOTIFICATION		
NotificationID	SERIAL	PRIMARY KEY
UserID	INT	NOT NULL, FK → USER
ReservationID	INT	NULL, FK → RESERVATION
Message	VARCHAR(255)	NOT NULL
Type	VARCHAR(30)	—
IsRead	BOOLEAN	DEFAULT FALSE
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

- No two rows in RESERVATION for the same ResourceID may have overlapping time ranges while both are Pending/Approved (no double booking).
- UNIQUE (ReservationID, ApproverID) in APPROVAL — the same approver reviews a given reservation only once.

Normalization and Functional Dependency Analysis

To ensure the database is free of redundancy and update anomalies, the design was normalized up to the Boyce-Codd Normal Form (BCNF).

- **1NF:** Initially, our conceptual booking register contained a repeating group of approval details per reservation. We resolved this by moving approvals into a separate APPROVAL relation, establishing an atomic structure where ReservationID and ApproverID serve as a composite primary key.
- **2NF:** The initial structure contained partial dependencies (e.g., ApproverName depended only on ApproverID, not the full composite key). Splitting the schema into dedicated RESERVATION and APPROVAL tables eliminated these.
- **3NF:** We identified and removed transitive dependencies. For example, ReservationID determined UserID, which in turn determined UserName and RoleID. By isolating user details into the USER table and resource details into the RESOURCE table, we satisfied 3NF.
- **BCNF:** Finally, an attribute closure analysis confirmed that for every relation (such as USER, ROLE, RESOURCE, and APPROVAL), every

determinant is a valid candidate key, confirming strict adherence to BCNF.

6. Database Selection & Justification

For this project, we selected **PostgreSQL** as our Database Management System. PostgreSQL was chosen over other relational databases like MySQL due to its advanced support for complex data types, robust transactional integrity, and superior concurrency control, which are absolutely critical for a scheduling application.

Justification Factors:

1. **Exclusion Constraints (btree_gist):** The most critical requirement of this project was preventing double bookings. PostgreSQL provides the btree_gist extension, which allows us to create exclusion constraints directly on time ranges (tsrange). This ensures the database strictly rejects any overlapping timestamps at the core level, avoiding race conditions that often occur when handling this logic in application code.

2. **Security (pgcrypto):** PostgreSQL's native pgcrypto extension allowed us to handle password hashing securely within the database itself using the crypt() function, minimizing the risk of exposing plain-text credentials.

3. **Advanced Triggers (PL/pgSQL):** The project required complex automated workflows, such as promoting a user from a waitlist when a slot frees up. PostgreSQL's procedural language (PL/pgSQL) made it straightforward to write sophisticated triggers to handle this background logic efficiently.

7. Implementation

The system's backend was implemented using a Python Flask application connected to the PostgreSQL database via the pg8000 DB-API driver. Below are

the critical SQL code snippets demonstrating the implementation of table creation, constraints, and business logic.

Constraint Implementation (Double-Booking Prevention):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE reservation (
  ReservationID SERIAL PRIMARY KEY,
  UserID INT NOT NULL REFERENCES users(UserID),
  ResourceID INT NOT NULL REFERENCES resource(ResourceID),
  BookingDate DATE NOT NULL,
  StartTime TIMESTAMP NOT NULL,
  EndTime TIMESTAMP NOT NULL CHECK (EndTime > StartTime),
  Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  -- Business Rule: Forbids overlapping bookings for the same resource
  EXCLUDE USING gist (ResourceID WITH =, tsrange(StartTime, EndTime) WITH &&)
          WHERE (Status IN ('Pending','Approved'))
);
```

Trigger Implementation (Maintenance Blocking):

This PL/pgSQL function actively prevents users from booking resources that are scheduled for maintenance

```
CREATE FUNCTION fn_block_maintenance() RETURNS trigger AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM maintenance_record m
             WHERE m.ResourceID = NEW.ResourceID
               AND m.Status IN ('Scheduled','In Progress')
               AND NEW.StartTime::date <= m.EndDate
               AND NEW.EndTime::date   >= m.StartDate) THEN
    RAISE EXCEPTION 'Resource is under maintenance for this period';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_block_maintenance BEFORE INSERT ON reservation
  FOR EACH ROW EXECUTE FUNCTION fn_block_maintenance();
```

View Implementation (Utilization Reporting):

This view aggregates data using joins to provide administrators with a clean report of resource usage.

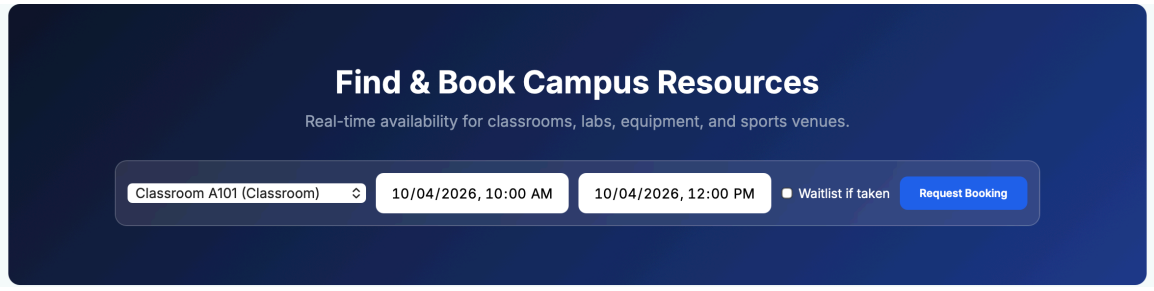
```sql
CREATE VIEW v_utilisation AS
SELECT r.ResourceID, r.ResourceName, c.CategoryName,
       COUNT(rv.ReservationID) AS approved_bookings,
       COALESCE(ROUND(SUM(EXTRACT(EPOCH FROM
       (rv.EndTime - rv.StartTime))/3600)::numeric, 1), 0) AS hours_booked
FROM resource r
JOIN resource_category c ON c.CategoryID = r.CategoryID
LEFT JOIN reservation rv ON rv.ResourceID = r.ResourceID AND rv.Status = 'Approved'
GROUP BY r.ResourceID, r.ResourceName, c.CategoryName;
```

8. Testing & Validation

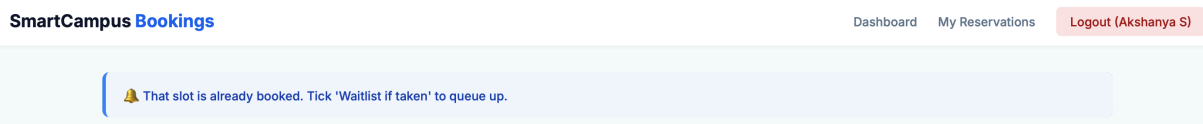
To ensure the integrity of our database constraints and application logic, we executed several test cases using the web frontend. The Flask application is designed to catch backend PostgreSQL exceptions and display them as alert messages on the user interface, proving that our database-level security and scheduling constraints are actively protecting the system.

Test Case 1: Overlapping Booking Prevention (Constraint Validation)

- **Description:** Verify that the database btree_gist exclusion constraint strictly stops a user from double-booking a slot.
- **Input / Action:** Log in as a student and attempt to book a resource for a time slot that is already taken by another user, without checking the "Waitlist" box.



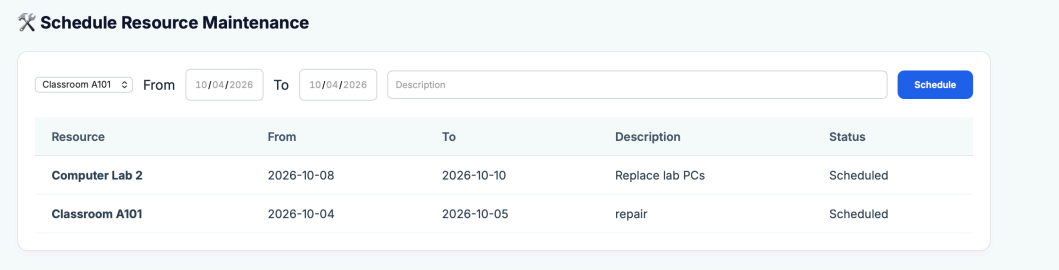
- **Expected Output:** The transaction is rejected at the database level by the exclusion constraint, and the UI displays a specific error message.
- **Actual Output:** The system flashes an alert: "That slot is already booked. Tick 'Waitlist if taken' to queue up.".

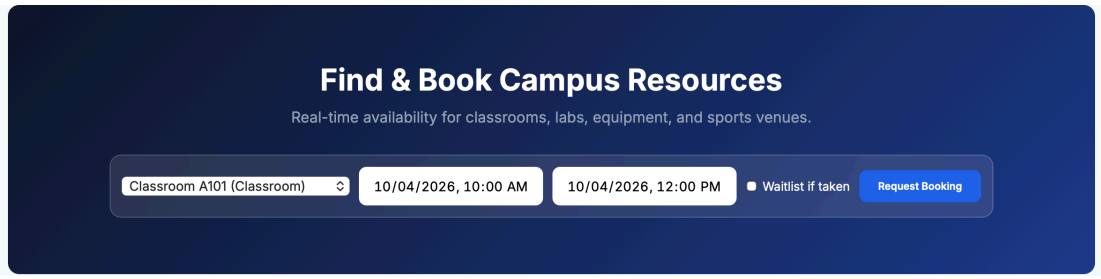


- **Status: PASS**

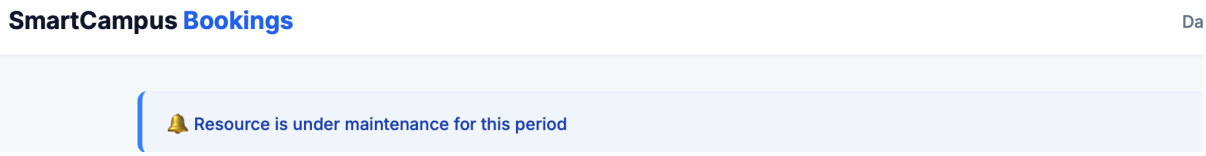
Test Case 2: Maintenance Block Validation (Trigger Validation)

- **Description:** Verify that the fn_block_maintenance database trigger successfully prevents users from booking resources actively scheduled for repair.
- **Input / Action:** As an Admin, schedule a maintenance block. Then, log in as a Student and attempt to book that exact resource during the maintenance dates.





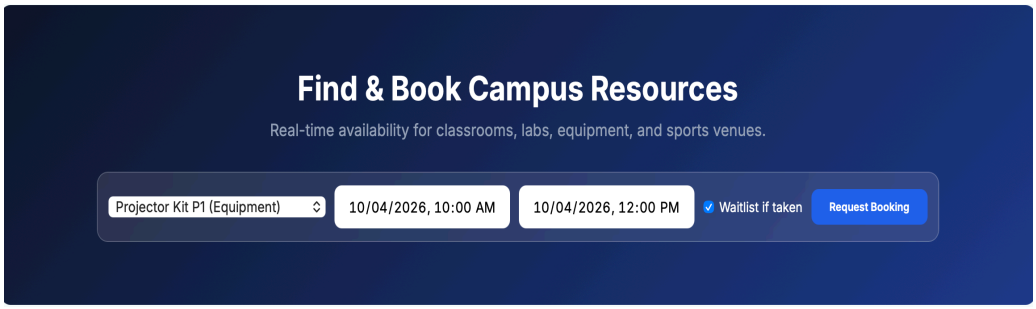
- **Expected Output:** The transaction is rejected by the PL/pgSQL database trigger, and the UI displays the custom PostgreSQL exception.
- **Actual Output:** The system flashes an alert: "Resource is under maintenance for this period.".



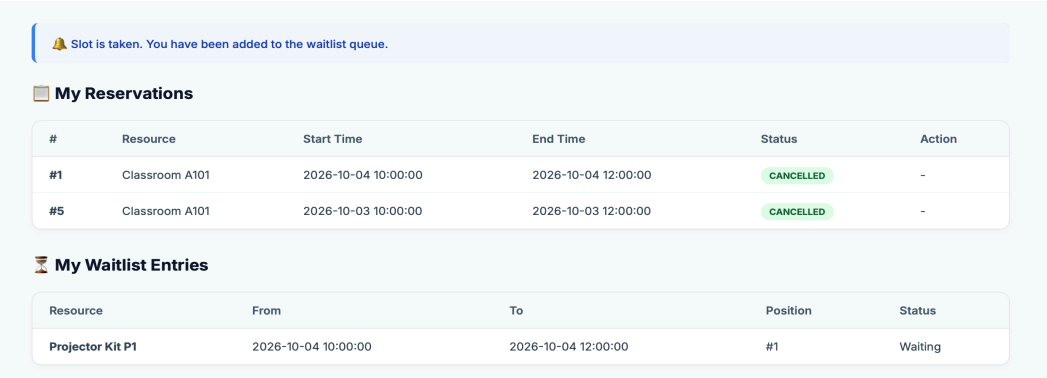
- **Status: PASS**

Test Case 3: Waitlist Queue Insertion Automation

- **Description:** Verify that the database properly routes a conflicting booking to the waitlist when requested by the user.
- **Input / Action:** Attempt to book a time slot that is already taken, but this time check the "Waitlist if taken" checkbox on the dashboard form.



- **Expected Output:** The system gracefully catches the 23P01 exclusion violation, bypasses the standard rejection, and successfully inserts the user into the WAITLIST table.
- **Actual Output:** The system flashes an alert: "Slot is taken. You have been added to the waitlist queue.".



- **Status: PASS**

9. Results & Screenshots

The implemented Smart Campus Resource Booking System successfully achieves all proposed objectives, providing a seamless interface backed by a robust database. Below are the key results demonstrating the functional application and underlying database execution.

1. Real-Time Resource Dashboard (User Interface)

This result demonstrates the core functionality of the frontend application. It showcases how the system retrieves data from the PostgreSQL backend to display the real-time availability of campus facilities (such as classrooms and labs) with clear visual status badges, allowing students to easily search and initiate bookings.

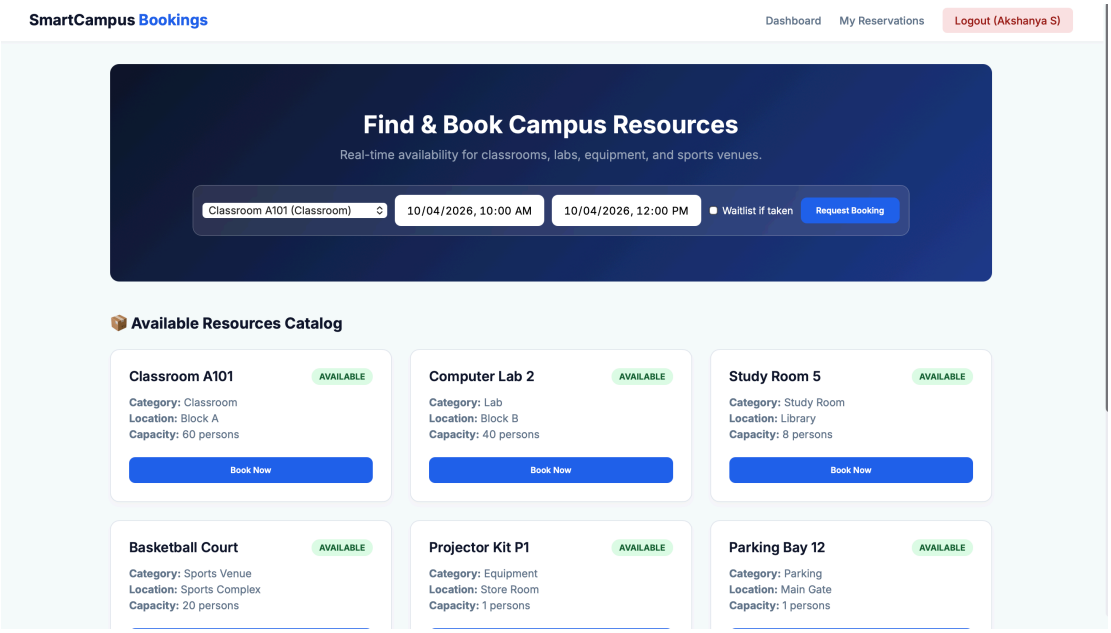


Figure 9.1 - The main student dashboard displaying categorized campus resources and real-time availability badges.

2. Administrative Utilization & Management (User Interface)

This result highlights the administrative capabilities of the system. It demonstrates the Admin Panel, specifically showcasing how the application queries the PostgreSQL v_utilisation view to generate an analytics report on resource usage, alongside the interface for handling pending reservation approvals

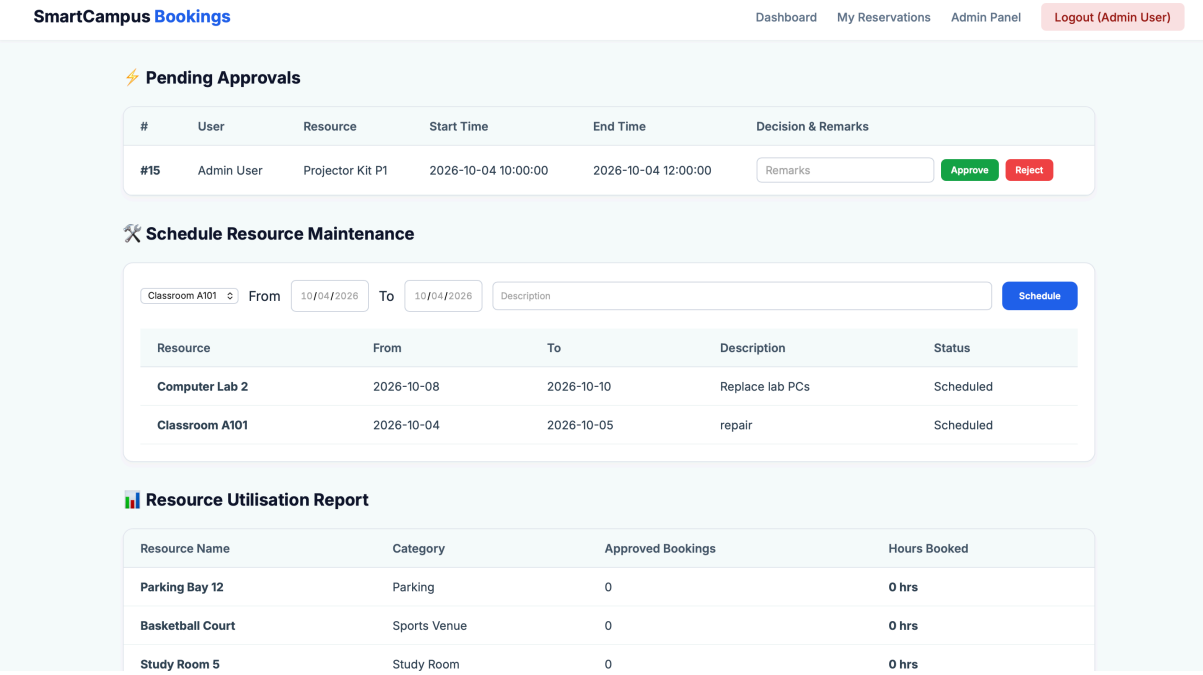


Figure 9.2 - The Admin Panel interface displaying pending reservations, maintenance scheduling, and the resource utilization report.

3. Cryptographic Data Security (pgAdmin 4 Backend)

To validate that the system protects sensitive user data, this result demonstrates the backend users table. It proves that the PostgreSQL pgcrypto extension is actively encrypting user passwords using the crypt() function, ensuring no plain-text credentials are stored in the database.

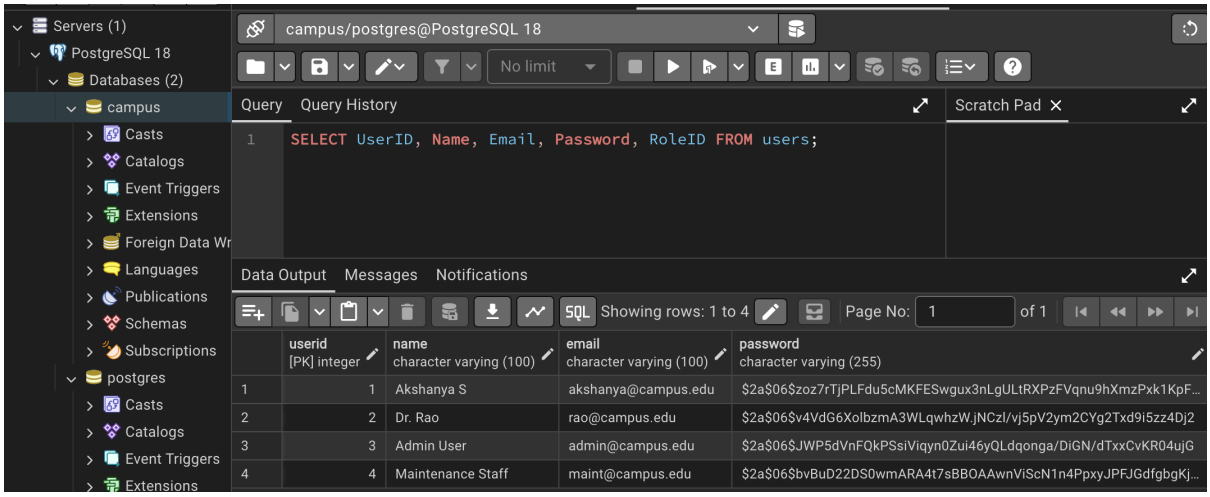


Figure 9.3 - Backend validation of data security showing passwords successfully hashed via the pgcrypto extension.

4. Complex Data Aggregation via Views (pgAdmin 4 Backend)

This result demonstrates the execution of complex backend data aggregation. It shows the output of the v_utilisation View directly inside the database, proving that the normalized relational schema successfully joins users, resources, and categories while calculating the total hours booked.

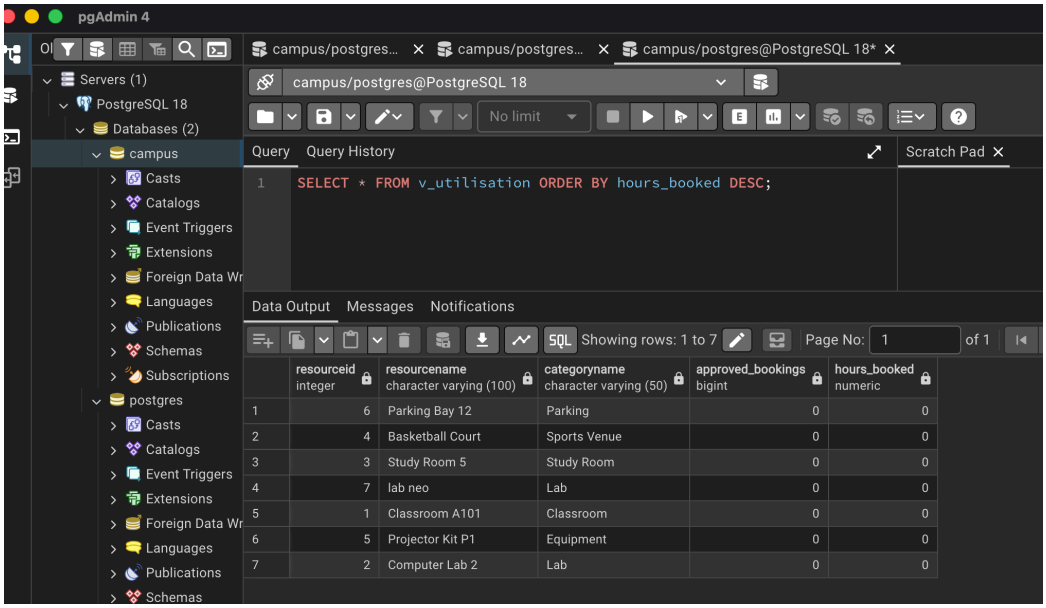


Figure 9.4 - pgAdmin 4 output of the utilization View demonstrating advanced multi-table JOINs and mathematical data aggregation

10. Challenges Faced & Solutions

During the development of the Smart Campus Resource Booking System, our team encountered several technical and design challenges, primarily centered around database concurrency, normalization, and trigger logic.

1. Database Design and Normalization (Handling Approvals)

- **Challenge:** Initially, designing the approval workflow presented a significant normalization issue. Since a reservation request could be reviewed, rejected, and later resubmitted, a single reservation could have multiple approval or rejection records associated with different administrators over time. Leaving this in the main reservation table created repeating groups, which directly violated the First Normal Form (1NF).

- **Solution:** We addressed this by decoupling the approval process into a dedicated APPROVAL entity. By establishing a composite primary key using (ReservationID, ApproverID), we effectively eliminated the repeating groups and partial dependencies, ensuring our final relational schema strictly adhered to BCNF.

2. Concurrency and Overlapping Booking Constraints

- **Challenge:** Preventing double bookings in a highly concurrent environment was a major hurdle. Initially, we attempted to validate overlapping times using SELECT queries in the Python Flask backend before executing an INSERT. However, we

realized this was highly vulnerable to race conditions if two users clicked "Book" at the exact same millisecond.

- **Solution:** We moved the validation entirely to the database tier. By implementing PostgreSQL's btree_gistextension, we applied an exclusion constraint using tsrange (timestamp ranges). This ensured the database engine inherently locked and rejected overlapping times, guaranteeing perfect data consistency without relying on application-level logic.

3. Automating Waitlist Management via Triggers

- **Challenge:** We wanted the system to automatically notify the next student in line if a booked slot was cancelled, rather than forcing administrators to monitor the waitlist manually. Handling this multi-step update via the web backend was inefficient and required multiple database trips.

- **Solution:** We developed a PL/pgSQL trigger (fn_waitlist_promote) that listens for an UPDATE on the reservation table. If a booking status changes to 'Cancelled' or 'Rejected', the trigger automatically queries the waitlist table, finds the user with Position 1 for that exact time slot, updates their status to 'Notified', and automatically inserts an alert into the notification table—all within a single database transaction.

11. Conclusion & Future Enhancements

Conclusion

The Smart Campus Resource Booking System successfully addresses the operational inefficiencies, scheduling conflicts, and lack of transparency inherent in manual campus facility management. By adhering to strict normalization principles up to BCNF, we developed a centralized platform that guarantees accurate, real-time availability tracking for all users.

The implementation heavily leverages advanced PostgreSQL features—such as GiST exclusion constraints to prevent double bookings, PL/pgSQL triggers to automate waitlist promotions and maintenance blocks, and cryptographic functions (pgcrypto) to secure user credentials. By integrating this robust relational database with a responsive web frontend, the project successfully meets all outlined objectives, ensuring a seamless, conflict-free reservation experience for the campus community.

Future Enhancements

While the current system is fully functional, several enhancements could further expand its utility:

1. **Mobile Application & Push Notifications:** Developing a dedicated mobile application would allow students to receive push notifications for waitlist promotions instantly, rather than relying on the web dashboard.

2. **Predictive Analytics for Resource Allocation:** By leveraging historical booking data accumulated in the database, machine learning algorithms could be applied to predict peak utilization times, helping administrators justify funding for new labs or study spaces.

3. **Hardware Integration (Smart Locks & ID Scanners):** Integrating the database with physical campus ID scanners would allow the system to automatically update a reservation to 'Completed' when a user taps into a room, or automatically cancel it if they fail to show up within 15 minutes.

4. **Cloud Deployment:** Migrating the PostgreSQL database and Flask application to a managed cloud platform (such as AWS RDS) would ensure higher availability and scalability during high-traffic periods, such as final exam weeks.

References

- **PostgreSQL Official Documentation** (https://www.postgresql.org/docs/) - Used extensively for the relational schema syntax, writing the PL/pgSQL triggers, and understanding the btree_gist extension for exclusion constraints.

- **Flask Web Framework Docs** (https://flask.palletsprojects.com/) - Used as the reference for building the Python backend and handling the web routes.

- **pg8000 Python Driver** (https://github.com/tlocke/pg8000) - Used for setting up the database connection between our Flask app and the PostgreSQL server.

- **MDN Web Docs** (https://developer.mozilla.org/) - Used as a general reference for HTML and CSS properties while building the responsive UI frontend

- **Gemini AI** (https://gemini.google.com/) - Used as a development tool to generate the initial HTML/CSS boilerplate code for the user interface based on our abstract

- **GitHub** (https://github.com/Dharun18-07/Smart_campus_system) - Used for version control throughout the project and hosting our final submission

- **23CSE202 Course Materials** - DBMS lecture slides, lab handouts, and project guidelines provided by the Amrita School of Computing, Amritapuri Campus, which guided our ER modeling and BCNF normalization